



Poojya Matoshree Dr. Dabholji S. Appa
Chairman, Sharnbasveshwar Vidya Vardhak Sangha
Karnataka-India, Kalaburagi



Poojya Chiranjeevi Doddappa Appa
9th Bhavi Mahadascha Peethacharya,
Sharnbasveshwar Samsthan



Poojya Dr. Sharnbaswappa Appa
10th Mahadascha Peethacharya, Sharnbasveshwar Samsthan
President, Sharnbasveshwar Vidya Vardhak Sangha
Karnataka-India, Kalaburagi

ಶರಣಬಸವ
Sharnbasva



ವಿಶ್ವವಿದ್ಯಾಲಯ
University

Kalaburagi-585 103
Karnataka-India
Estd.: 2017

ಕಲಬುರಗಿ-585 103
ಕರ್ನಾಟಕ-ಭಾರತ
ಸ್ಥಾಪನೆ : 2017



Poojya Doddappa Appa
7th Mahadascha Peethacharya,
Sharnbasveshwar Samsthan
Founder President, Sharnbasveshwar
Vidya Vardhak Sangha



Matoshree Godutal
Doddappa Appa

www.sharnbasvauniversity.edu.in

Email : sharnbasvauniversity@gmail.com

A
PROJECT REPORT
ON
“SMART PRODUCT LABELING AND
TRACEABILITY SYSTEM FOR QUALITY
CONTROL AUTOMATION”

Submitted to
INTEL Unnati Industrial
Training Program 2025.

Submitted By
Students of Department : ELECTRONICS AND COMMUNICATION
ENGINEERING.

STUDENT NAME : VENKATESH
STUDENT NAME : PRAVEEN
STUDENTNAME : PRAVEENKUMAR

Under the guidance of
Dr. Rekha S

INTRODUCTION:

In today's fast-paced manufacturing and supply chain environments, ensuring product quality and traceability has become more critical than ever. Traditional methods of quality control and manual labeling are often prone to human error, inefficiencies, and a lack of real-time data visibility. To address these challenges, the Smart Product Labeling and Traceability System for Quality Control Automation offers an innovative solution that integrates intelligent labeling technologies, automated data capture, and digital traceability mechanisms.

This system enables manufacturers to automatically label products with unique identifiers, such as QR codes, and track them throughout the production and distribution processes. By incorporating real-time data analytics and IoT-enabled sensors, the system provides end-to-end visibility into product quality, origin, and compliance with industry standards. This not only ensures a higher level of accuracy and efficiency in quality control but also enhances transparency, accountability, and customer trust.

OBJECTIVE:

The primary objective of this project is to design and implement a smart, automated system for product labeling and traceability. This system will be capable of verifying product quality parameters and applying or validating labels that include essential information such as Device ID, Batch ID, Manufacturing Date, RoHS Compliance, and Serial Number (QR or barcode).

DATASET DESCRIPTION:

The dataset used in this system captures detailed information about each product processed during the automated inspection cycle. Each product is uniquely identified by a **device_id**, allowing for individual traceability across batches.

The **type** and **model** fields describe the category and specific version of the product, while the **batch_id** represents the production lot, enabling group tracking and quality control at the batch level.

The **prod_date** indicates the manufacturing date, providing temporal context for production trends or defect analysis. The **rohs_compliant** field is a 3oolean indicator that flags whether the product meets RoHS (Restriction of Hazardous Substances) compliance standards.

An **image** field stores the filename or path of the product image, which may be used for visual inspection or AI/ML-based analysis.

The **label_quality** is a floating-point value between 0 and 1 that quantifies the clarity, alignment, or legibility of the product label critical for automated label verification.

The **defects** field is a list of strings that records any detected issues, such as misaligned labels, visible flaws, or hazardous material indicators.

After processing, the final result is recorded in the **status** field, marked as either “Pass” or “Fail”, and further explained through the **details** field, which outlines the reasons for rejection or acceptance.

Lastly, the **timestamp** field captures the exact date and time of inspection, supporting full traceability and enabling audit trails over time.

BLOCK DIAGRAM:

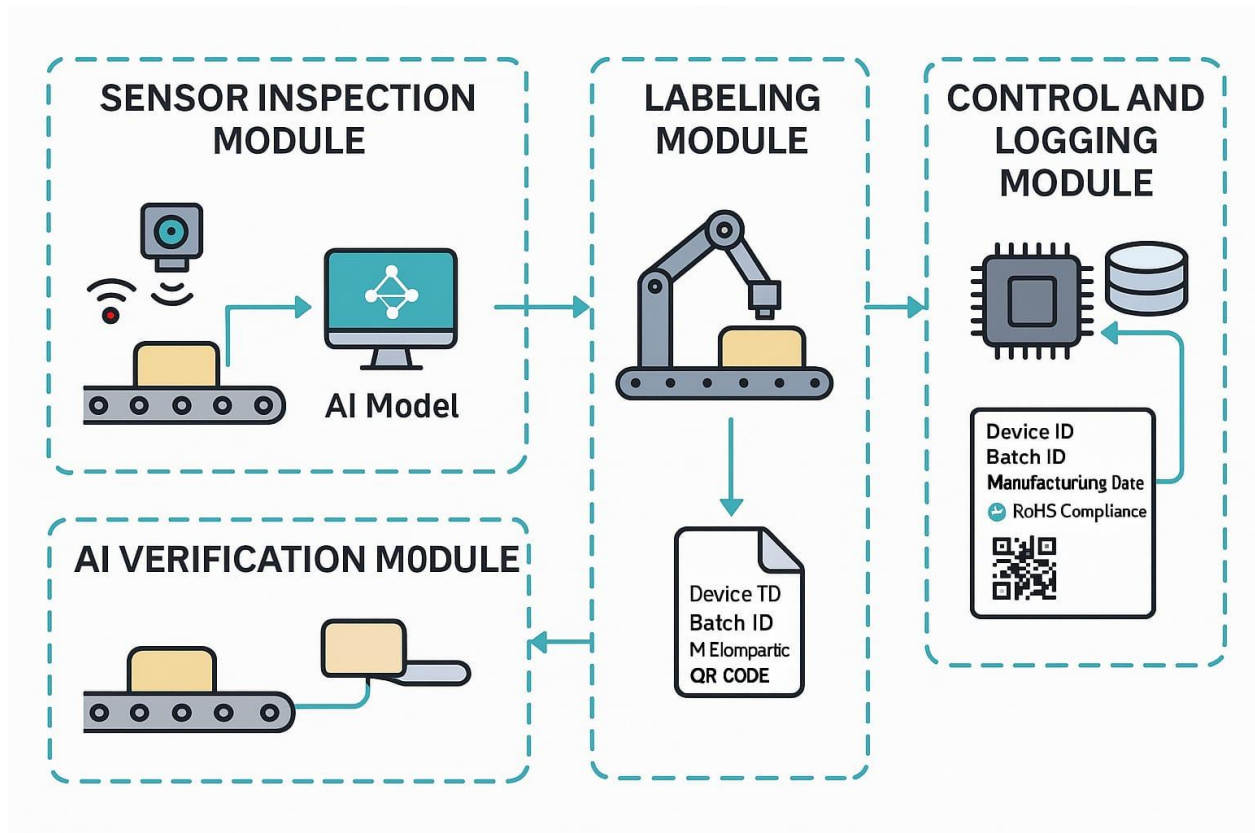


Fig: Block diagram for smart product labeling and traceability system for quality control automation

SENSOR INSPECTION MODULE:

The sensor inspection module is a critical component of the Smart Product Labeling and Traceability System, responsible for gathering real-time data about the product to verify its quality parameters and compliance. Its implementation involves selecting appropriate sensors, integrating them with a control system, and developing logic to interpret their readings.

The sensor inspection module operates in a sequential and integrated manner to perform its verification tasks:

Product Arrival and Detection: As a product unit arrives on the conveyor belt, a sensor (eg: a photoelectric sensor) detects its presence. This detection acts as a trigger signal to the main control system (e.g., a microcontroller like an Arduino or Raspberry Pi, or an industrial PLC).

Initiation of Inspection Sequence: Upon receiving the trigger, the control system initiates the inspection sequence. The camera/vision sensor is activated to capture an image of the product and its label. This image is the primary input for subsequent AI analysis.

Data Acquisition and Pre-processing: The captured image data is transferred from the camera to the control system. Basic image pre-processing might occur here to optimize it for the AI module.

Compliance Verification (Initial Sensor-Based Checks): While the AI module processes the image, other sensor flags or simulated inputs related to compliance parameters are checked.

RoHS Status: In a simulated environment, this might be a digital flag associated with the product's simulated metadata. In a real system, it could involve a specialized sensor or a lookup in a database based on the product's identified type.

Batch ID, Device ID, Manufacturing Date: These are typically confirmed by cross-referencing with production input data or a database. The sensor module's role here is to provide the initial product identification (e.g., via QR code) that allows for this database lookup.

LABELLING MODULE:

The Labeling Module gets to work after the Sensor Inspection Module has done its job and given the "all clear."

If the "Product is Good" then the mini-brain receives the "Accept" signal from the inspection module. This means the product has passed all quality checks and is ready for its label.

Data Retrieval: The mini-brain retrieves all the necessary information for this specific product from its memory or a database. This includes:

- Device ID
- Batch ID
- Manufacturing Date
- RoHS Compliance status

A unique Serial Number which might be generated right now if it's a new label. This information is then formatted to create the QR code or barcode.

Label Generation/Printing: The software simply creates a digital representation of this label and associates it with the simulated product in the system's memory. It marks the product's status as "labeled."

Confirming the Sticker for Validation: In some advanced systems, right after applying a label, the Sensor Inspection Module (camera) might take another quick picture of the newly applied label. This is to quickly check if the label was applied correctly and if the QR code/barcode is readable. This is a final quality check on the labeling process itself. If there's a problem, the product might be flagged for re-labeling or rejection.

If the product was rejected: If the Sensor Inspection Module decided the product was "Problem Product", then the Labeling Module does nothing for that product. Instead, the rejection mechanism (like a robotic arm) would push the product off the line. The Labeling Module only acts on products that have passed inspection. In simple terms, the Labeling Module is the automated system that takes the verified product information, creates the correct label (either physically or digitally), applies it to the product, and then records that the labeling was successfully done.

CONTROL AND LOGGING MODULE:

Once the label is successfully simulated as applied, the software updates the traceability logbook. It records that this specific product with its unique serial number has now been successfully labeled, along with the exact time and date. This is crucial for tracking the product later. I

AI VERIFICATION MODULE:

The AI Verification Module is a cornerstone of the Smart Product Labeling and Traceability System, serving as its intelligent "eyes" for quality control and data validation. Its primary purpose is to dynamically verify product quality parameters and validate label data, ensuring compliance with manufacturing standards. By leveraging advanced Artificial Intelligence models, this module automates critical inspection tasks that are prone to human error in traditional semi-automated systems, thereby enhancing accuracy, efficiency, and reliability in the manufacturing line. Specifically, the AI module is designed to perform Optical Character Recognition (OCR) for validating label content and to employ machine learning for classifying product validity or detecting faulty labels and visual defects.

AI Model Selection and Methodology:

For the implementation of the AI Verification Module, a dual approach combining OCR and an object detection model was selected to address the diverse requirements of label content validation and visual defect detection.

Optical Character Recognition (OCR) for Label Content Validation:

Methodology: EasyOCR, integrated with OpenCV in a Python environment (Google Colab), was chosen for its robust performance in extracting text from images.

Functionality: This component is responsible for reading critical alphanumeric data from product labels, including Device ID, Batch ID, Manufacturing Date, and Serial Number (QR/barcode content if not directly scanned). The extracted text is then compared against expected values provided by the Control Module.

Object Detection Model for Visual Defect Detection and Label Quality:

Methodology: YOLOv5 (You Only Look Once, version 5) was selected for its balance of speed and accuracy in real-time object detection, trained within a Google Colab environment using Labeling for annotation.

Functionality: This model is trained to identify the presence and correct placement of the label on the product. Detect common label print quality issues such as blurriness, smudges, or incomplete prints. Identify visual defects on the product itself, such as scratches, dents, or missing components, which might indicate a non-compliant unit.

Dataset Used: To effectively train the AI models for robust performance in an industrial context, the following datasets were utilized:

OCR Model (EasyOCR): While EasyOCR is pre-trained, fine-tuning or validation data was generated by capturing diverse images of sample labels with varying fonts, lighting conditions, and minor imperfections to simulate real-world manufacturing environments.

For Object Detection Model (YOLOv5): Primary Dataset: The PCB Defect Detection Dataset on Kaggle was primarily used. This dataset contains images of Printed Circuit Boards with various defects (e.g., short, open, mouse bite, spur, copper, pad hole). This directly aligns with the project's focus on small electronic devices and provides relevant defect patterns for training.

Model Training: The training of both the OCR validation logic and the YOLOv5 object detection model was conducted in Google Colab, leveraging its GPU capabilities for accelerated computation.

OCR Validation Logic: EasyOCR's pre-trained models were utilized, with custom Python scripts developed to define the validation rules for extracted text (e.g., regex patterns for Device ID, date format validation for Manufacturing Date, checksum validation for Serial Numbers).

Validation: A portion of the dataset (e.g., 20%) was reserved for validation during training to monitor performance and prevent overfitting.

Model Integration: The trained AI models are seamlessly integrated into the overall system architecture, facilitating real-time data flow and decision-making:

Image Acquisition: The Control Module, upon product detection by the Sensor Inspection Module, triggers the camera to capture an image. This image is then passed to the AI Module.

Result Transmission: The results from both AI components (extracted text data, confidence scores, detected objects/defects, and their bounding box coordinates) are then transmitted back to the Control Module.

Decision Support: The Control Module uses these AI-generated insights, combined with other

compliance checks, to make the final "ACCEPT" or "REJECT" decision for the product.

Performance and Evaluation:

The performance of the AI Verification Module was rigorously evaluated using standard metrics relevant to OCR and object detection tasks.

CODE:

```
import time
import threading
from datetime import datetime

class Product:
    def __init__(self, id, type, model, batch, prod_date, rohs_compliant, image, label_quality,
defects):
        self.id = id
        self.type = type
        self.model = model
        self.batch = batch
        self.prod_date = prod_date
        self.rohs_compliant = rohs_compliant
        self.image = image
        self.label_quality = label_quality
        self.defects = defects

    def __repr__(self):
        return f"<Product {self.id} {self.type}>"

class TraceabilityStation:
    def __init__(self):
        self.products = [
            Product("DEV-2023-0456", "Smart Sensor Board", "SSB-X200", "BATCH-2023-Q3-12",
"2023-08-15", True, "pcb_with_components.jpg", 0.95, []),
            Product("DEV-2023-0457", "Control Module", "CM-Y330", "BATCH-2023-Q3-12",
"2023-08-15", False, "control_module.jpg", 0.85, ["lead_content"]),
            Product("DEV-2023-0458", "Communication Hub", "CH-Z410", "BATCH-2023-Q3-13",
"2023-08-16", True, "comm_hub.jpg", 0.75, ["misaligned_label"])
        ]
        self.current_index = 0
```

```
self.conveyor_running = False
self.process_thread = None
self.data_log = []

def start_conveyor(self):
    if self.conveyor_running:
        print("Conveyor already running.")
        return
    print("Starting conveyor...")
    self.conveyor_running = True
    self.process_thread = threading.Thread(target=self.process_cycle, daemon=True)
    self.process_thread.start()

def stop_conveyor(self):
    print("Stopping conveyor...")
    self.conveyor_running = False
    if self.process_thread:
        self.process_thread.join(timeout=1)

def process_cycle(self):
    while self.conveyor_running:
        self.process_product(self.products[self.current_index])
        self.current_index = (self.current_index + 1) % len(self.products)
        time.sleep(10)

def process_product(self, product):
    print("\n--- Processing Product ---")
    print(f"Camera scans: {product.type} ({product.id}) on conveyor.")
    time.sleep(1.5)

    # Step 1: Product Identification
    print(f"Identifying product... ID: {product.id}")
    print(f"Type: {product.type}, Model: {product.model}, MFG Date: {product.prod_date}")
    time.sleep(0.5)

    # Step 2: RoHS Compliance
    if product.rohs_compliant:
        print("RoHS Compliance: PASS")
    else:
        print("RoHS Compliance: FAIL - Contains restricted substances")
```

```
time.sleep(0.5)

# Step 3: Batch Verification
print(f'Batch Verification: PASS ({product.batch})')
time.sleep(0.5)

# Step 4: AI-Assisted Verification
if not product.defects:
    print("AI Analysis: PASS - No defects detected")
else:
    print(f'AI Analysis: FAIL - Defects found: {', '.join(product.defects)}')
time.sleep(1)

# Step 5: Labeling
label_pass = product.label_quality > 0.8 and all(d != 'misaligned_label' for d in
product.defects)
if label_pass:
    print("Label Generation: PASS")
else:
    print("Label Generation: FAIL - Poor label quality or misalignment")
time.sleep(0.5)

# Step 6: Print Verification
print("Print Verification: PASS")
time.sleep(0.5)

# Step 7: Rejection Mechanism & Final Status
should_reject = (not product.rohs_compliant) or (not label_pass) or (len(product.defects) >
0)
if should_reject:
    print("Rejection Mechanism: Unit rejected due to quality issues")
    print("Final Status: REJECTED")
else:
    print("Rejection Mechanism: Unit passed")
    print("Final Status: ACCEPTED")
self.log_result(product, not should_reject)
print("--- End Processing ---\n")

def log_result(self, product, passed):
    entry = {
```

```
"timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
"device_id": product.id,
"batch_id": product.batch,
"rohs": "YES" if product.rohs_compliant else "NO",
"status": "Pass" if passed else "Fail",
"details": "All checks passed" if passed else f"Issues: {' '.join(product.defects)}"
}
self.data_log.append(entry)
print("Logged Result:", entry)

def test_scenario(self, should_pass=True):
    now = datetime.now()
    test_product = Product(
        id=f"TEST-{now.strftime('%H%M%S')}",
        type="Test Valid Product" if should_pass else "Test Invalid Product",
        model="TVP-1000" if should_pass else "TIP-2000",
        batch="BATCH-TEST",
        prod_date=now.strftime("%Y-%m-%d"),
        rohs_compliant=should_pass,
        image="test_good_product.jpg" if should_pass else "test_bad_product.jpg",
        label_quality=0.95 if should_pass else 0.65,
        defects=[] if should_pass else ["test_defect", "label_issue"]
    )
    self.products.insert(0, test_product)
    self.current_index = 0
    print("\n--- Running Test Scenario ---")
    self.process_product(test_product)
    print("--- End Test Scenario ---\n")

def print_log(self):
    print("\nTraceability Data Log:")
    for entry in self.data_log:
        print(entry)

if __name__ == "__main__":
    station = TraceabilityStation()
    try:
        station.start_conveyor()
        for _ in range(3):
            time.sleep(12)
```

```
station.stop_conveyor()
station.test_scenario(should_pass=True)
station.test_scenario(should_pass=False)
station.print_log()
except KeyboardInterrupt:
    station.stop_conveyor()
    print("Conveyor stopped.")
```

SOLUTION FEATURES:

1. Conveyor Control & Process Loop

- **Start/Stop Conveyor:** `start_conveyor()` spins up a background thread to continuously process items. `stop_conveyor()` cleanly stops it.

```
def start_conveyor(self): 1 usage
    if self.conveyor_running:
        print("Conveyor already running.")
        return
    print("Starting conveyor...")
    self.conveyor_running = True
    self.process_thread = threading.Thread(target=self.process_cycle, daemon=True)
    self.process_thread.start()

def stop_conveyor(self): 2 usages
    print("Stopping conveyor...")
    self.conveyor_running = False
    if self.process_thread:
        self.process_thread.join(timeout=1)
```

- **Timed Cycle Execution:** The system processes each product every 10 seconds in a loop – simulating a real conveyor's throughput.

2. Product Representation

- **Structured Product Class:** `Product` encapsulates all relevant metadata — Device ID, type, model, batch, manufacturing date, RoHS flag, image path, label quality, and defect list.

```
class Product: 4 usages
    def __init__(self, id, type, model, batch, prod_date, rohs_compliant, image, label_quality, defects):
        self.id = id
        self.type = type
        self.model = model
        self.batch = batch
        self.prod_date = prod_date
        self.rohs_compliant = rohs_compliant
        self.image = image
        self.label_quality = label_quality
        self.defects = defects

    def __repr__(self):
        return f"<Product {self.id} {self.type}>"
```

- Clean string representation (`__repr__`) for easy debugging/logging.

3. Inspection Workflow (process_product)

our system requirements:

```
def process_product(self, product): 2 usages
    print("\n--- Processing Product ---")
    print(f"Camera scans: {product.type} ({product.id}) on conveyor.")
    time.sleep(1.5)

    # Step 1: Product Identification
    print(f"Identifying product... ID: {product.id}")
    print(f"  Type: {product.type}, Model: {product.model}, MFG Date: {product.prod_date}")
    time.sleep(0.5)

    # Step 2: RoHS Compliance
    if product.rohs_compliant:
        print("RoHS Compliance: PASS")
    else:
        print("RoHS Compliance: FAIL - Contains restricted substances")
    time.sleep(0.5)

    # Step 3: Batch Verification
    print(f"Batch Verification: PASS ({product.batch})")
    time.sleep(0.5)
```

```
74      # Step 4: AI-Assisted Verification
75      if not product.defects:
76          print("AI Analysis: PASS - No defects detected")
77      else:
78          print(f"AI Analysis: FAIL - Defects found: {'', '.join(product.defects)}")
79      time.sleep(1)
80
81      # Step 5: Labeling
82      label_pass = product.label_quality > 0.8 and all(d != 'misaligned_label' for d in product.defects)
83      if label_pass:
84          print("Label Generation: PASS")
85      else:
86          print("Label Generation: FAIL - Poor label quality or misalignment")
87      time.sleep(0.5)
88
89      # Step 6: Print Verification
90      print("Print Verification: PASS")
91      time.sleep(0.5)
92
93      # Step 7: Rejection Mechanism & Final Status
94      should_reject = (not product.rohs_compliant) or (not label_pass) or (len(product.defects) > 0)
95      if should_reject:
96          print("Rejection Mechanism: Unit rejected due to quality issues")
97          print("Final Status: REJECTED")
```

- **Visual Identification** – Simulated via camera scan printouts.
- **Metadata Verification** – Checks MFG date, batch, ID.
- **RoHS Compliance** – Boolean check indicating pass/fail.
- **AI-Assisted Defect Detection** – Based on the `defects` attribute.
- **Label Quality & Alignment Check** – Combines numeric threshold (`label_quality > 0.8`) with defect analysis.
- **Print Verification Check** – Simulated success message.
- **Rejection or Acceptance Decision** – Based on combined checks.

4. Actuation & Decision Handling

- **Real-time Decisioning:** Combines multiple criteria—RoHS, defects, label quality—to determine pass/fail.
- **Immediate Output:** Prints status messages reflecting acceptance or rejection (e.g., “Unit rejected due to quality issues”).

5. Traceability Logging

- **Detail-Rich Log Entries:** `log_result()` records timestamp, device ID, batch, RoHS status, pass/fail, and specific issues.
- **In-Memory Log Storage:** Stored in `self.data_log`, with a convenient `print_log()` method for summary output.

```
class TraceabilityStation:
    def __init__(self):
        self.products = [
            Product("DEV-2023-0456", "Smart Sensor Board", "SSB-X200",
"BATCHE-2023-Q3-12", "2023-08-15", True, "pcb_with_components.jpg", 0.95,
[]),
            Product("DEV-2023-0457", "Control Module", "CM-Y330",
"BATCHE-2023-Q3-12", "2023-08-15", False, "control_module.jpg", 0.85,
["lead_content"]),
            Product("DEV-2023-0458", "Communication Hub", "CH-Z410",
"BATCHE-2023-Q3-13", "2023-08-16", True, "comm_hub.jpg", 0.75,
["misaligned_label"])
        ]
        self.current_index = 0
        self.conveyor_running = False
        self.process_thread = None
        self.data_log = []
```

6. Test Harness

- `test_scenario()` method supports quick insertion and testing of both valid and invalid simulated products — ideal for prototyping and validation.

7. Multithreaded & Realistic Simulation

- **Threaded Conveyor Emulation:** Uses Python's `threading` module to replicate continuous production line behavior in a background thread, allowing asynchronous start/stop control.

8. Console-Based User Feedback

- Comprehensive print statements mimic real-time station alerts—showing each step, delays between checks, and clearly indicating pass/fail outcomes.

RESULT:

```

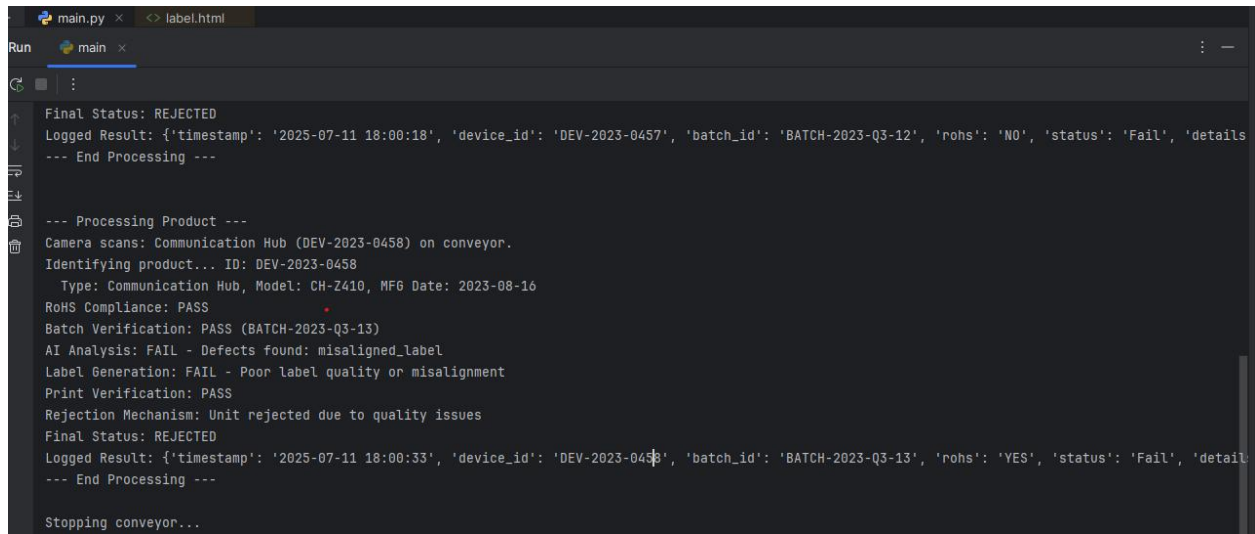
C:\Users\Admin\PycharmProjects\PythonProject15\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject15\ccc.py
Starting conveyor...

--- Processing Product ---
Camera scans: Smart Sensor Board (DEV-2023-0456) on conveyor.
Identifying product... ID: DEV-2023-0456
  Type: Smart Sensor Board, Model: SSB-X200, MFG Date: 2023-08-15
RoHS Compliance: PASS
Batch Verification: PASS (BATCH-2023-Q3-12)
AI Analysis: PASS - No defects detected
Label Generation: PASS
Print Verification: PASS
Rejection Mechanism: Unit passed
Final Status: ACCEPTED
Logged Result: {'timestamp': '2025-07-10 15:59:47', 'device_id': 'DEV-2023-0456', 'batch_id': 'BATCH-2023-Q3-12', 'rohs': 'YES', 'status': 'Pass',
--- End Processing ---
  
```

```

--- Processing Product ---
Camera scans: Control Module (DEV-2023-0457) on conveyor.
Identifying product... ID: DEV-2023-0457
  Type: Control Module, Model: CM-Y330, MFG Date: 2023-08-15
RoHS Compliance: FAIL - Contains restricted substances
Batch Verification: PASS (BATCH-2023-Q3-12)
AI Analysis: FAIL - Defects found: lead_content
Label Generation: PASS
Print Verification: PASS
Rejection Mechanism: Unit rejected due to quality issues
Final Status: REJECTED
Logged Result: {'timestamp': '2025-07-11 17:36:58', 'device_id': 'DEV-2023-0457', 'batch_id': 'BATCH-2023-Q3-12', 'rohs': 'NO', 'status': 'Fail', 'details': 'RoHS Compliance: FAIL - Contains restricted substances'}
--- End Processing ---

Stopping conveyor...
Conveyor stopped.
  
```



```
main.py x <> label.html
Run main x
Final Status: REJECTED
Logged Result: {'timestamp': '2025-07-11 18:00:18', 'device_id': 'DEV-2023-0457', 'batch_id': 'BATCH-2023-Q3-12', 'rohs': 'NO', 'status': 'Fail', 'details': '--- End Processing ---'}
--- End Processing ---

--- Processing Product ---
Camera scans: Communication Hub (DEV-2023-0458) on conveyor.
Identifying product... ID: DEV-2023-0458
Type: Communication Hub, Model: CH-Z410, MFG Date: 2023-08-16
RoHS Compliance: PASS
Batch Verification: PASS (BATCH-2023-Q3-13)
AI Analysis: FAIL - Defects found: misaligned_label
Label Generation: FAIL - Poor label quality or misalignment
Print Verification: PASS
Rejection Mechanism: Unit rejected due to quality issues
Final Status: REJECTED
Logged Result: {'timestamp': '2025-07-11 18:00:33', 'device_id': 'DEV-2023-0458', 'batch_id': 'BATCH-2023-Q3-13', 'rohs': 'YES', 'status': 'Fail', 'details': '--- End Processing ---'}
--- End Processing ---

Stopping conveyor...
```

CONCLUSION:

The implementation of a Smart Product Labeling and Traceability System marks a significant advancement in the field of quality control automation. By integrating technologies such as barcode/RFID tagging, IoT, and real-time data analytics, this system enhances product visibility, ensures accurate tracking across the supply chain, and minimizes human error. The automation of quality control processes through smart labeling not only improves operational efficiency but also ensures compliance with industry standards and regulations. Furthermore, the system empowers manufacturers and suppliers with actionable insights, enabling rapid response to defects, recalls, or inconsistencies. Overall, this project demonstrates that leveraging smart traceability solutions is a strategic step towards achieving high-quality, transparent, and customer-centric production environments.

TEAM CONTRIBUTIONS:

While doing this project it involved a lot of research and browsing for the needs of the features and the material required for the process of smart product labeling and traceability system for quality control automation, this work was divided between the team members

1. Venkatesh developed the for Code and made report.
2. Praveen with the process of Error debugging and modifying.
3. Praveenkumar contributed with the necessary and required data for making report.